In [2]:
```
import pandas as pd
import numpy as np

# Load the dataset
print("Loading data...")
tb_data = pd.read_csv("C:\\Users\\pc\\Downloads\\Tuberculosis_Trends.csv")

# Initial inspection
print("\nData Overview:")
print(f"Dataset shape: {tb_data.shape}")
print("\nFirst 3 rows:")
print(tb_data.head(3))
print("\nData types:")
print(tb_data.dtypes)
print("\nSummary statistics:")
print(tb_data.describe())
```

```
Loading data...

Data Overview:
Dataset shape: (3000, 22)

First 3 rows:
     Country         Region  Income_Level  Year  TB_Cases  TB_Deaths  \
0  Indonesia           Asia  Upper-Middle  2002     14128       3846
1     Brazil  North America  Lower-Middle  2017      1069        342
2    Nigeria  South America          High  2006      2473       6316

   TB_Incidence_Rate  TB_Mortality_Rate  TB_Treatment_Success_Rate  \
0             150.58              18.99                      54.52
1             167.80              44.20                      88.19
2             285.77               6.52                      64.55

   Drug_Resistant_TB_Cases  ...  GDP_Per_Capita  \
0                     3977  ...           25516
1                     4179  ...           26300
2                     4961  ...           30771

   Health_Expenditure_Per_Capita  Urban_Population_Percentage  \
0                           3180                        50.49
1                           5529                        24.06
2                           1703                        81.77

   Malnutrition_Prevalence  Smoking_Prevalence  TB_Doctors_Per_100K  \
0                    28.84               22.90                 4.10
1                    28.65               33.79                 8.18
2                    38.69                6.63                 2.26

   TB_Hospitals_Per_Million  Access_To_Health_Services  \
0                     15.46                      52.38
1                      3.06                      46.95
2                      4.95                      40.88

   BCG_Vaccination_Coverage  HIV_Testing_Coverage
0                     95.87                 88.39
1                     63.78                 35.60
2                     68.91                 77.81

[3 rows x 22 columns]

Data types:
Country                          object
Region                           object
Income_Level                     object
Year                              int64
TB_Cases                          int64
TB_Deaths                         int64
TB_Incidence_Rate               float64
TB_Mortality_Rate               float64
TB_Treatment_Success_Rate       float64
Drug_Resistant_TB_Cases           int64
HIV_CoInfected_TB_Cases           int64
Population                        int64
```

```
GDP_Per_Capita                    int64
Health_Expenditure_Per_Capita     int64
Urban_Population_Percentage     float64
Malnutrition_Prevalence         float64
Smoking_Prevalence              float64
TB_Doctors_Per_100K             float64
TB_Hospitals_Per_Million        float64
Access_To_Health_Services       float64
BCG_Vaccination_Coverage        float64
HIV_Testing_Coverage            float64
dtype: object

Summary statistics:
              Year      TB_Cases    TB_Deaths  TB_Incidence_Rate  \
count  3000.000000   3000.000000  3000.000000        3000.000000
mean   2012.049000  24807.937667  5090.818333         255.845483
std       7.247306  14261.279606  2846.753032         140.651939
min    2000.000000    515.000000    50.000000          10.060000
25%    2006.000000  12297.750000  2660.500000         133.662500
50%    2012.000000  24467.000000  5188.000000         255.770000
75%    2018.000000  37031.500000  7512.500000         374.647500
max    2024.000000  49985.000000  9991.000000         499.950000

       TB_Mortality_Rate  TB_Treatment_Success_Rate  Drug_Resistant_TB_Cases  \
count        3000.000000                3000.000000              3000.000000
mean           25.119210                  72.484597              2501.796333
std            14.031132                  13.112578              1440.032384
min             1.010000                  50.000000                13.000000
25%            12.962500                  61.067500              1236.000000
50%            25.090000                  72.545000              2525.500000
75%            36.900000                  84.025000              3740.000000
max            49.990000                  94.980000              4999.000000

       HIV_CoInfected_TB_Cases    Population  GDP_Per_Capita  \
count              3000.000000  3.000000e+03     3000.000000
mean               4929.499333  7.036535e+08    30046.892333
std                2846.943009  4.018102e+08    17172.813451
min                  10.000000  1.088106e+06      501.000000
25%                2520.750000  3.636371e+08    15212.000000
50%                4864.000000  6.970638e+08    29707.000000
75%                7328.500000  1.041607e+09    44463.000000
max                9989.000000  1.399285e+09    59996.000000

       Health_Expenditure_Per_Capita  Urban_Population_Percentage  \
count                    3000.000000                  3000.000000
mean                     5039.485667                    54.558710
std                      2870.269590                    20.124572
min                        50.000000                    20.000000
25%                      2543.500000                    37.067500
50%                      5008.000000                    54.775000
75%                      7518.750000                    71.575000
max                      9999.000000                    89.990000

       Malnutrition_Prevalence  Smoking_Prevalence  TB_Doctors_Per_100K  \
count              3000.000000         3000.000000          3000.000000
mean                 27.549927           22.439453             5.015230
```

```
std                  12.974430          10.179438             2.851124
min                   5.010000           5.020000             0.100000
25%                  16.385000          13.750000             2.520000
50%                  27.635000          22.195000             4.930000
75%                  38.782500          31.522500             7.480000
max                  49.970000          40.000000            10.000000

       TB_Hospitals_Per_Million  Access_To_Health_Services  \
count                3000.000000                3000.000000
mean                   10.268077                  64.597103
std                     5.717438                  19.990994
min                     0.510000                  30.010000
25%                     5.130000                  47.240000
50%                    10.390000                  64.630000
75%                    15.360000                  81.342500
max                    20.000000                  99.960000

       BCG_Vaccination_Coverage  HIV_Testing_Coverage
count                3000.000000           3000.000000
mean                   74.588197             55.498690
std                    14.124815             20.250306
min                    50.020000             20.000000
25%                    62.517500             38.240000
50%                    74.870000             55.825000
75%                    86.645000             73.497500
max                    98.990000             89.980000
```

In [3]:
```python
# Define authoritative country-region mapping
print("\nCorrecting regional data...")
region_mapping = {
    'Brazil': 'South America',
    'USA': 'North America',
    'Russia': 'Europe',
    'China': 'Asia',
    'India': 'Asia',
    'Indonesia': 'Asia',
    'Pakistan': 'Asia',
    'Nigeria': 'Africa',
    'South Africa': 'Africa',
    'Bangladesh': 'Asia'
}

# Before correction
print("\nRegion counts before correction:")
print(tb_data['Region'].value_counts())

# Apply corrections
tb_data['Region'] = tb_data['Country'].map(region_mapping).fillna(tb_data['Region']

# After correction
print("\nRegion counts after correction:")
print(tb_data['Region'].value_counts())

# Verify corrections
print("\nUnique country-region pairs:")
print(tb_data[['Country', 'Region']].drop_duplicates().sort_values('Country'))
```

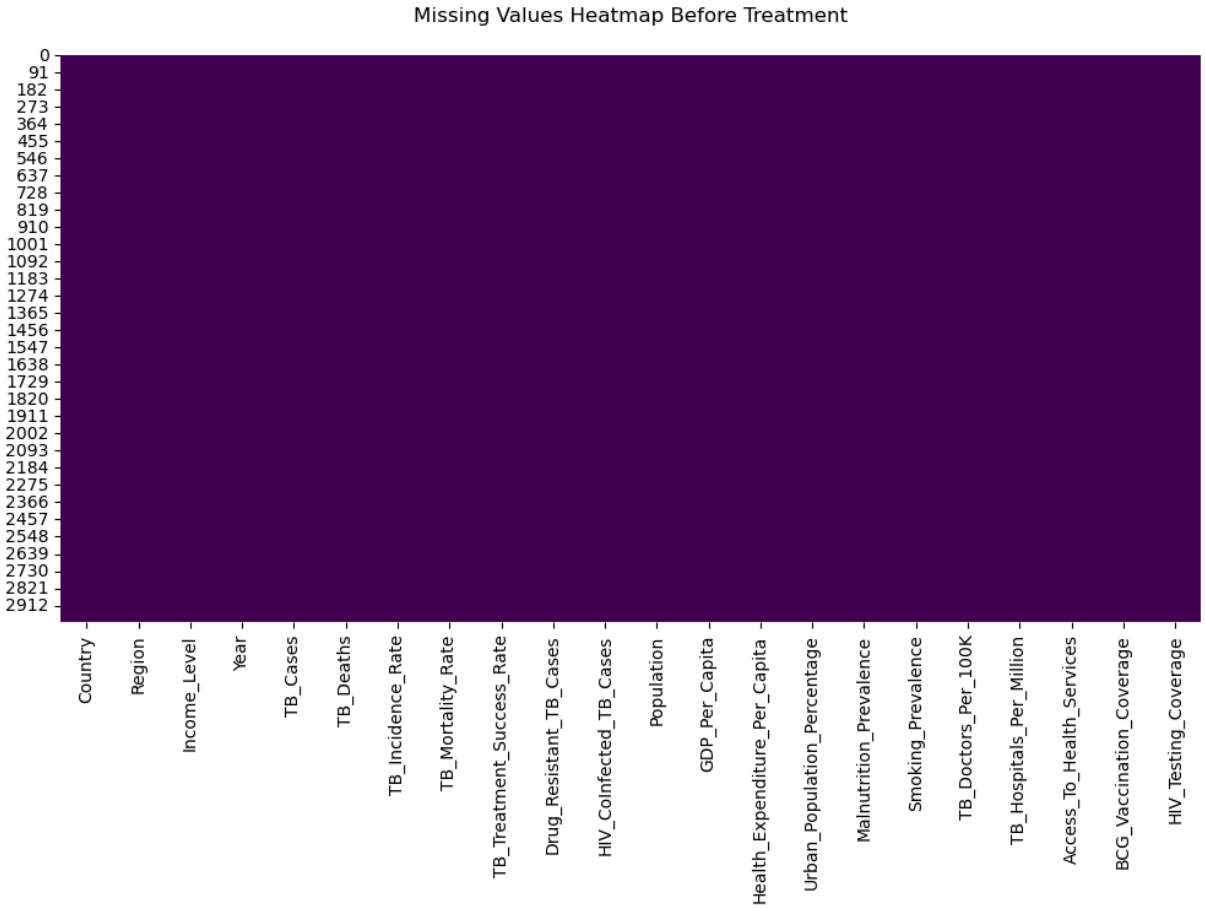
```
Correcting regional data...

Region counts before correction:
Region
Africa           625
South America    611
North America    594
Asia             590
Europe           580
Name: count, dtype: int64

Region counts after correction:
Region
Asia             1488
Africa            595
Europe            315
North America     311
South America     291
Name: count, dtype: int64

Unique country-region pairs:
         Country         Region
5     Bangladesh           Asia
1         Brazil  South America
16         China           Asia
21         India           Asia
0      Indonesia           Asia
2        Nigeria         Africa
24      Pakistan           Asia
3         Russia         Europe
14  South Africa         Africa
6            USA  North America
```

In [4]:

```python
# Missing value analysis
print("\nMissing value analysis:")
missing_values = tb_data.isnull().sum()
missing_percent = (tb_data.isnull().sum() / len(tb_data)) * 100
missing_report = pd.concat([missing_values, missing_percent], axis=1)
missing_report.columns = ['Missing Count', 'Missing %']
print(missing_report[missing_report['Missing Count'] > 0].sort_values('Missing %',

# Visualize missing values
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.heatmap(tb_data.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Values Heatmap Before Treatment', pad=20)
plt.show()

# Handle missing values
print("\nTreating missing values...")

# Numerical columns - median imputation
num_cols = tb_data.select_dtypes(include=np.number).columns
for col in num_cols:
```

```
    if tb_data[col].isnull().sum() > 0:
        median_val = tb_data[col].median()
        tb_data[col].fillna(median_val, inplace=True)
        print(f"Filled {missing_values[col]} missing values in {col} with median: {

# Categorical columns - mode imputation
cat_cols = tb_data.select_dtypes(include='object').columns
for col in cat_cols:
    if tb_data[col].isnull().sum() > 0:
        mode_val = tb_data[col].mode()[0]
        tb_data[col].fillna(mode_val, inplace=True)
        print(f"Filled {missing_values[col]} missing values in {col} with mode: '{m

# Verify no missing values remain
print("\nMissing values after treatment:", tb_data.isnull().sum().sum())
```

```
Missing value analysis:
Empty DataFrame
Columns: [Missing Count, Missing %]
Index: []
```




```
Treating missing values...

Missing values after treatment: 0
```

In [5]:
```
print("\nPerforming data quality checks...")

# Check for impossible values in key metrics
def check_value_ranges(df, col, min_val, max_val):
    invalid = df[(df[col] < min_val) | (df[col] > max_val)]
```

```
    if not invalid.empty:
        print(f"Warning: {len(invalid)} rows with {col} outside expected range ({mi
        return invalid
    else:
        print(f"{col} values all within expected range ({min_val}-{max_val})")
        return None

# Validate key columns
check_value_ranges(tb_data, 'TB_Mortality_Rate', 0, 500)
check_value_ranges(tb_data, 'TB_Incidence_Rate', 0, 1000)
check_value_ranges(tb_data, 'TB_Treatment_Success_Rate', 0, 100)

# Check for duplicate rows
duplicates = tb_data.duplicated()
print(f"\nFound {duplicates.sum()} duplicate rows")
if duplicates.sum() > 0:
    tb_data = tb_data.drop_duplicates()
    print("Duplicate rows removed")

# Check year range
print("\nYear range:", tb_data['Year'].min(), "to", tb_data['Year'].max())

# Final data shape
print("\nFinal cleaned data shape:", tb_data.shape)
```

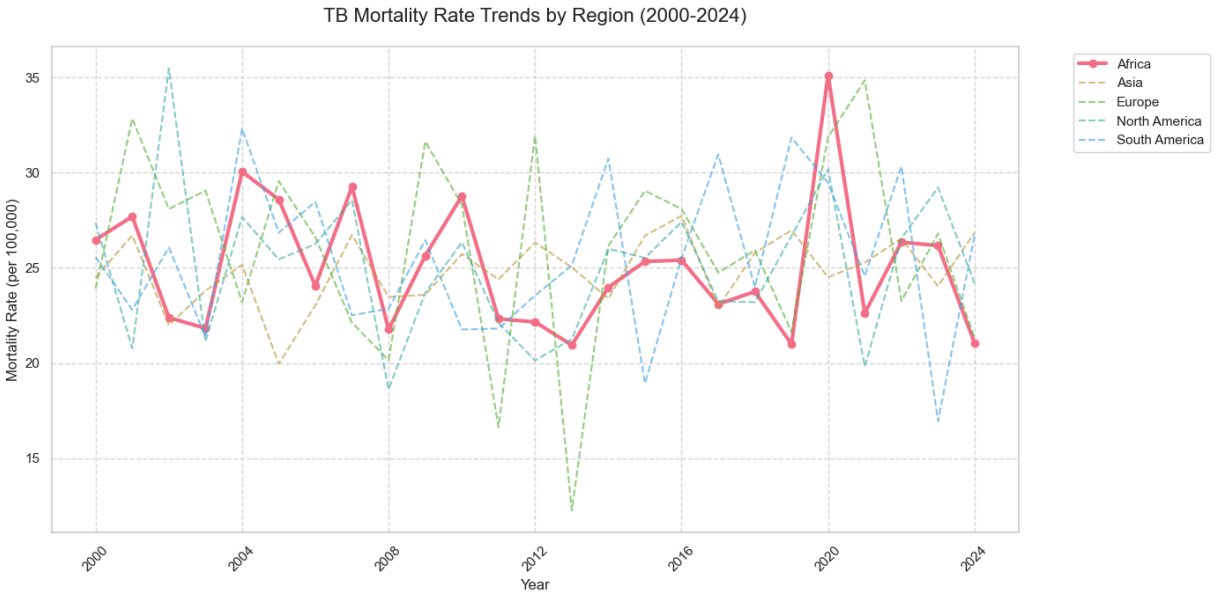
```
Performing data quality checks...
TB_Mortality_Rate values all within expected range (0-500)
TB_Incidence_Rate values all within expected range (0-1000)
TB_Treatment_Success_Rate values all within expected range (0-100)

Found 0 duplicate rows

Year range: 2000 to 2024

Final cleaned data shape: (3000, 22)
```

In [6]:

```
# Save cleaned data
clean_filename = 'TB_Data_Cleaned.csv'
tb_data.to_csv(clean_filename, index=False)
print(f"\nCleaned data saved to {clean_filename}")

# Save cleaning report
with open('data_cleaning_report.txt', 'w') as f:
    f.write("TB Data Cleaning Report\n")
    f.write("=======================\n")
    f.write(f"Original shape: {tb_data.shape}\n")
    f.write(f"Missing values treated: {missing_values.sum()}\n")
    f.write("Regional corrections applied to:\n")
    for country, region in region_mapping.items():
        f.write(f"- {country}: {region}\n")
    f.write("\nFinal data checks passed:\n")
    f.write("- No remaining missing values\n")
    f.write("- All values within expected ranges\n")
    f.write(f"- {duplicates.sum()} duplicate rows removed\n")

print("Cleaning report saved to data_cleaning_report.txt")
```

```
Cleaned data saved to TB_Data_Cleaned.csv
Cleaning report saved to data_cleaning_report.txt
```

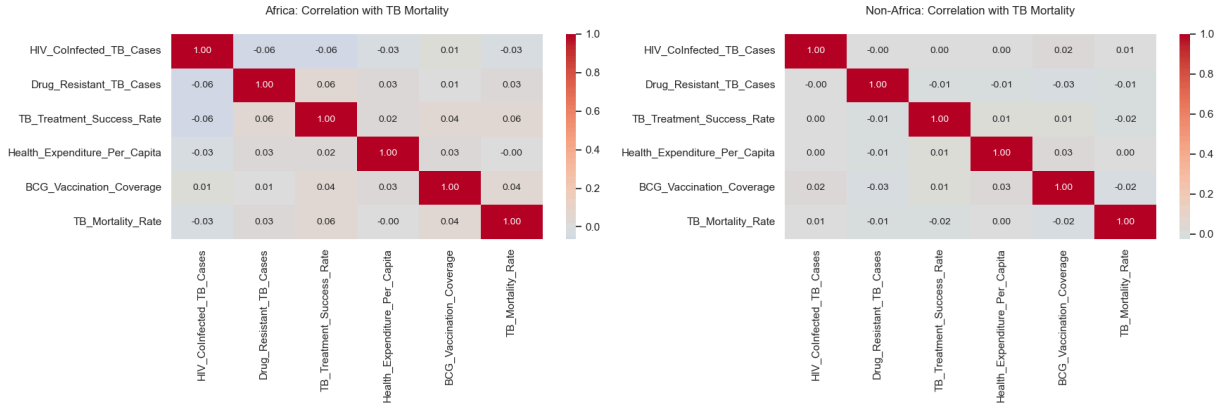
In [9]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Set professional style using modern approach
sns.set_theme(style="whitegrid")  # or "darkgrid", "ticks", "dark"
sns.set_palette("husl")

# Convert Year to datetime for better plotting
tb_data['Year'] = pd.to_datetime(tb_data['Year'], format='%Y')

# Calculate regional mortality trends
regional_trends = tb_data.groupby(['Region', 'Year'])['TB_Mortality_Rate'].mean().u

# Plot mortality trends
plt.figure(figsize=(14, 7))
plt.plot(regional_trends.index, regional_trends['Africa'],
         label='Africa', linewidth=3, marker='o')

for region in regional_trends.columns:
    if region != 'Africa':
        plt.plot(regional_trends.index, regional_trends[region],
                 label=region, alpha=0.6, linestyle='--')

plt.title('TB Mortality Rate Trends by Region (2000-2024)', fontsize=16, pad=20)
plt.ylabel('Mortality Rate (per 100,000)', fontsize=12)
plt.xlabel('Year', fontsize=12)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Calculate percentage changes
trend_comparison = pd.DataFrame({
    '2000': regional_trends.iloc[0],
    '2024': regional_trends.iloc[-1],
    'Change (%)': (regional_trends.iloc[-1] - regional_trends.iloc[0]) / regional_t
}).sort_values('Change (%)')

print("\nMortality Rate Change (2000-2024):")
print(trend_comparison[['2000', '2024', 'Change (%)']].round(1))
```




```
Mortality Rate Change (2000-2024):
               2000  2024  Change (%)
Region
Africa         26.5  21.0       -20.5
North America  27.4  24.2       -11.6
Europe         23.9  21.3       -10.9
South America  25.6  26.7         4.5
Asia           24.5  26.8         9.6
```

In [10]:
```
# Select key co-factors
co_factors = ['HIV_CoInfected_TB_Cases', 'Drug_Resistant_TB_Cases',
              'TB_Treatment_Success_Rate', 'Health_Expenditure_Per_Capita',
              'BCG_Vaccination_Coverage']

# Calculate correlations for Africa
africa_corr = tb_data[tb_data['Region'] == 'Africa'][co_factors + ['TB_Mortality_Ra

# Calculate correlations for non-Africa
non_africa_corr = tb_data[tb_data['Region'] != 'Africa'][co_factors + ['TB_Mortalit

# Visualize comparison
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))

sns.heatmap(africa_corr, annot=True, cmap='coolwarm', center=0,
            annot_kws={'size': 10}, fmt='.2f', ax=ax1)
ax1.set_title('Africa: Correlation with TB Mortality', pad=20)

sns.heatmap(non_africa_corr, annot=True, cmap='coolwarm', center=0,
            annot_kws={'size': 10}, fmt='.2f', ax=ax2)
ax2.set_title('Non-Africa: Correlation with TB Mortality', pad=20)

plt.tight_layout()
plt.show()

# Identify key differences
print("\nTop correlations with TB Mortality:")
print("In Africa:")
print(africa_corr['TB_Mortality_Rate'].abs().sort_values(ascending=False)[1:4])
```

```
print("\nOutside Africa:")
print(non_africa_corr['TB_Mortality_Rate'].abs().sort_values(ascending=False)[1:4])
```



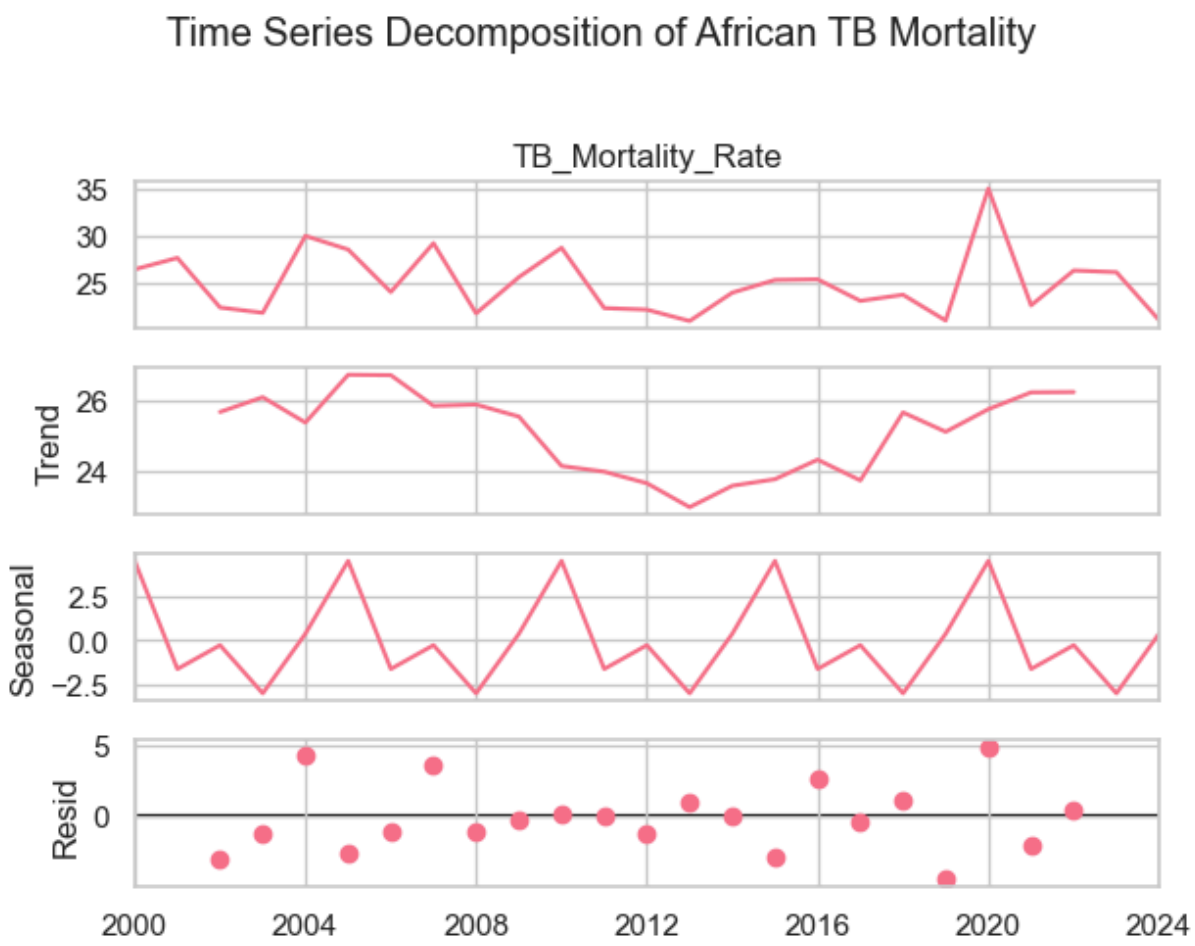

```
Top correlations with TB Mortality:
In Africa:
TB_Treatment_Success_Rate    0.060424
BCG_Vaccination_Coverage     0.037092
HIV_CoInfected_TB_Cases      0.033798
Name: TB_Mortality_Rate, dtype: float64

Outside Africa:
BCG_Vaccination_Coverage     0.024289
TB_Treatment_Success_Rate    0.016353
HIV_CoInfected_TB_Cases      0.008462
Name: TB_Mortality_Rate, dtype: float64
```

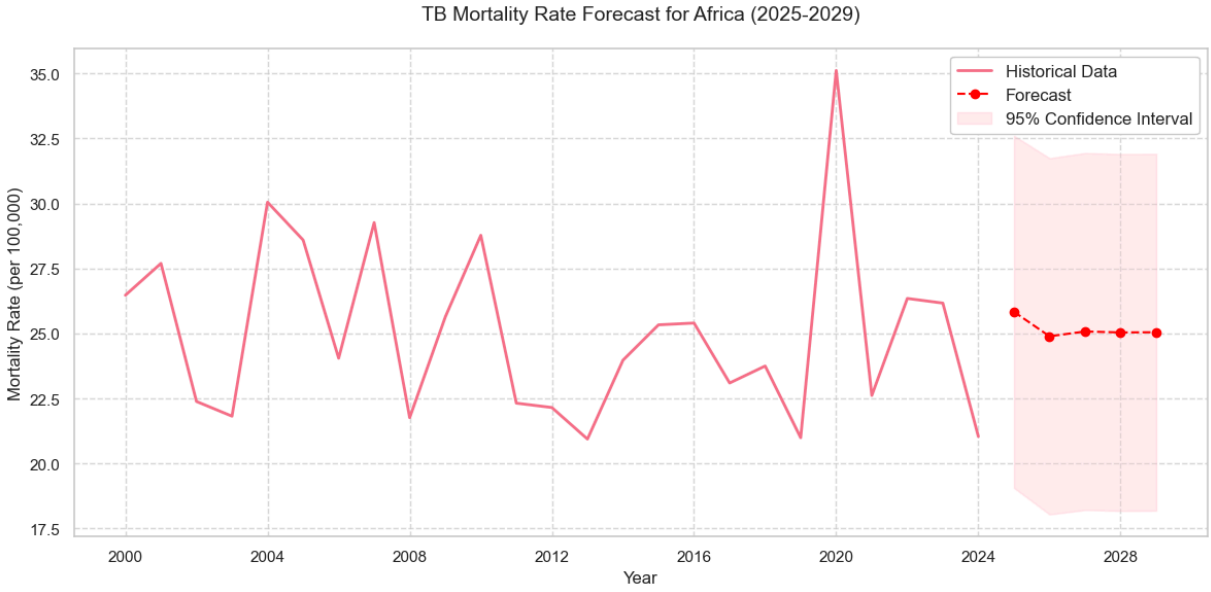
In [30]:

```
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error
import matplotlib.pyplot as plt
import pandas as pd

# Prepare Africa time series with explicit frequency
africa_ts = tb_data[tb_data['Region'] == 'Africa'].groupby('Year')['TB_Mortality_Ra

# Convert to proper time series with yearly frequency
africa_ts.index = pd.to_datetime(africa_ts.index, format='%Y')
africa_ts = africa_ts.asfreq('YS')  # Year-Start frequency

# Time series decomposition
plt.figure(figsize=(12, 8))
try:
    decomposition = seasonal_decompose(africa_ts, model='additive', period=5)
    decomposition.plot()
    plt.suptitle('Time Series Decomposition of African TB Mortality', y=1.02)
    plt.tight_layout()
    plt.show()
except ValueError as e:
    print(f"Decomposition failed: {e}")

# ARIMA modeling with proper frequency handling
try:
    model = ARIMA(africa_ts, order=(1,1,1))
    model_fit = model.fit()
```

```
    # Forecast next 5 years
    forecast = model_fit.get_forecast(steps=5)
    forecast_values = forecast.predicted_mean
    conf_int = forecast.conf_int()

    # Plot forecast with improved styling
    plt.figure(figsize=(12, 6))
    plt.plot(africa_ts.index, africa_ts, label='Historical Data', linewidth=2)
    plt.plot(forecast_values.index, forecast_values, color='red',
             linestyle='--', marker='o', label='Forecast')
    plt.fill_between(conf_int.index,
                     conf_int.iloc[:,0],
                     conf_int.iloc[:,1],
                     color='pink', alpha=0.3, label='95% Confidence Interval')

    plt.title('TB Mortality Rate Forecast for Africa (2025-2029)', fontsize=14, pad
    plt.ylabel('Mortality Rate (per 100,000)', fontsize=12)
    plt.xlabel('Year', fontsize=12)
    plt.legend(fontsize=12, framealpha=1)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

    # Print forecast values with more context
    print("\n5-Year Mortality Rate Forecast for Africa:")
    forecast_df = pd.DataFrame({
        'Year': forecast_values.index.year,
        'Forecast': forecast_values.round(2),
        'Lower CI': conf_int.iloc[:,0].round(2),
        'Upper CI': conf_int.iloc[:,1].round(2)
    })
    print(forecast_df.to_string(index=False))

except Exception as e:
    print(f"ARIMA modeling failed: {e}")
```

```
<Figure size 1200x800 with 0 Axes>
```





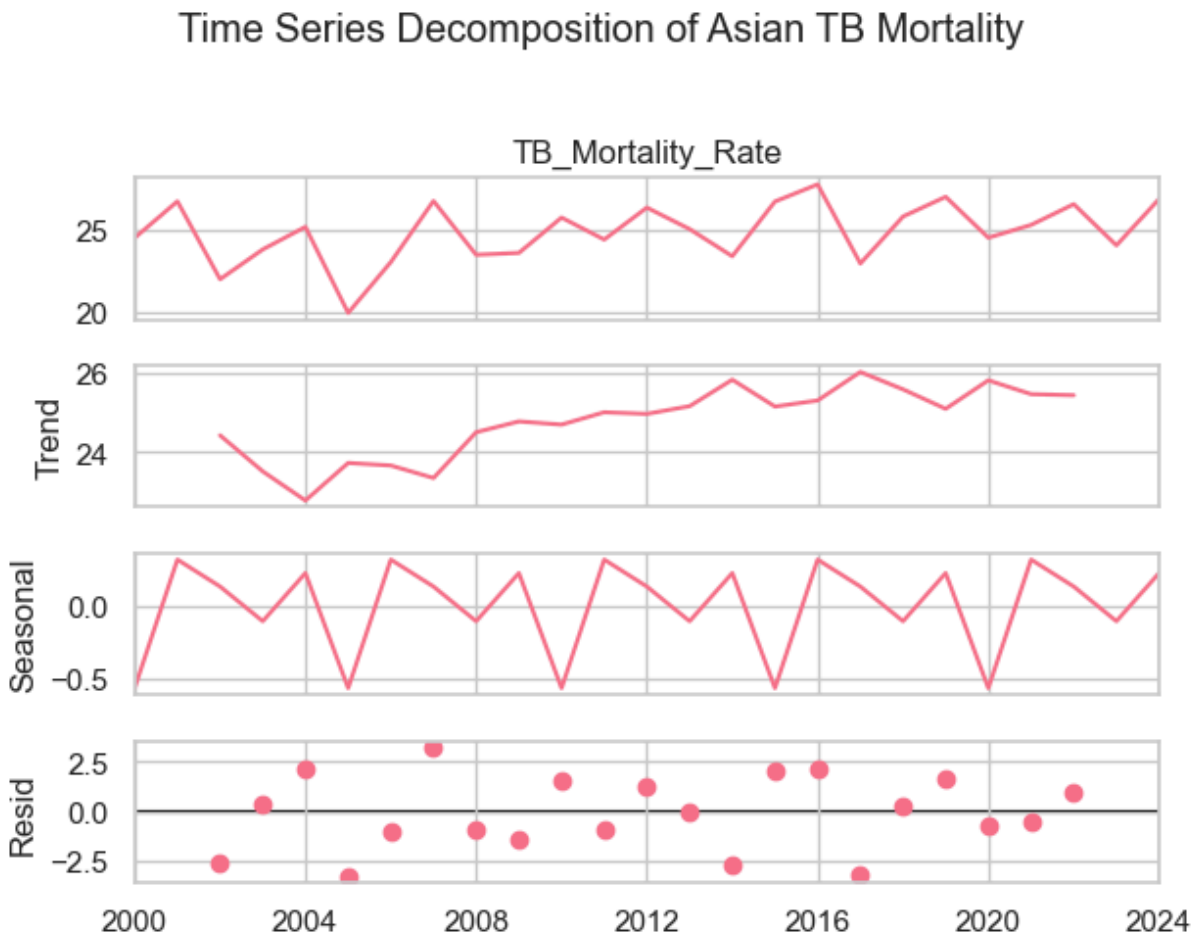

```
5-Year Mortality Rate Forecast for Africa:
 Year  Forecast  Lower CI  Upper CI
 2025     25.83     19.07     32.60
 2026     24.89     18.05     31.74
 2027     25.08     18.22     31.94
 2028     25.04     18.18     31.90
 2029     25.05     18.19     31.91
```

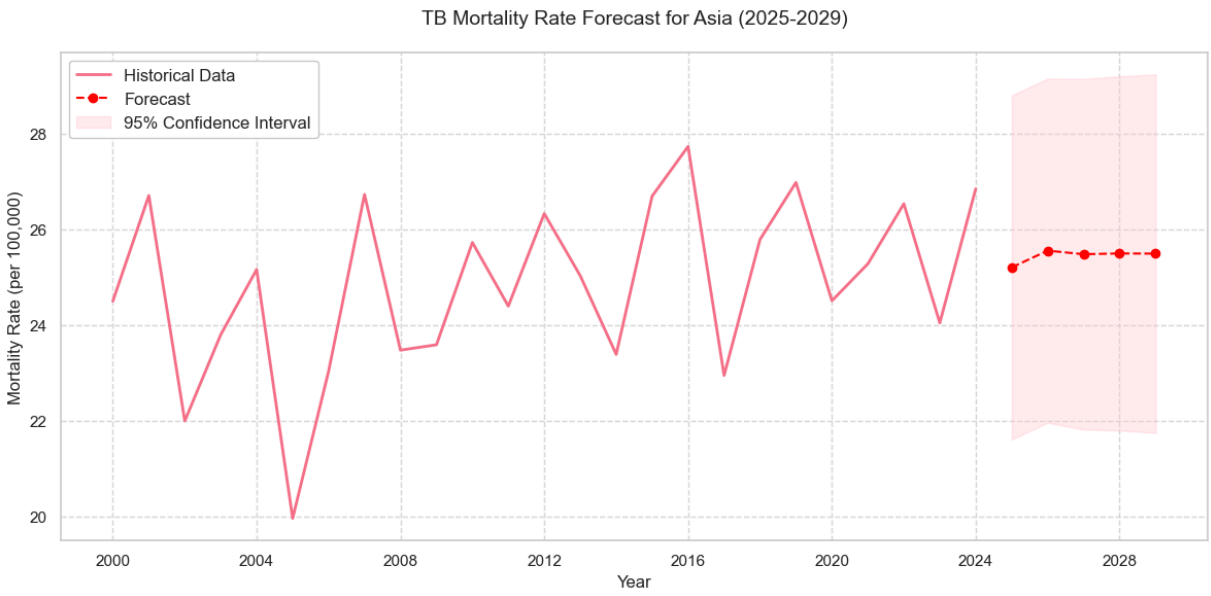
In [14]:
```
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
```

```
import matplotlib.pyplot as plt
import pandas as pd

# Prepare Asia time series with explicit frequency
asia_ts = tb_data[tb_data['Region'] == 'Asia'].groupby('Year')['TB_Mortality_Rate']

# Convert to proper time series with yearly frequency
asia_ts.index = pd.to_datetime(asia_ts.index, format='%Y')
asia_ts = asia_ts.asfreq('YS')  # Year-Start frequency

# Time series decomposition
plt.figure(figsize=(12, 8))
try:
    decomposition = seasonal_decompose(asia_ts, model='additive', period=5)
    decomposition.plot()
    plt.suptitle('Time Series Decomposition of Asian TB Mortality', y=1.02)
    plt.tight_layout()
    plt.show()
except ValueError as e:
    print(f"Decomposition failed: {e}")

# ARIMA modeling with proper frequency handling
try:
    model = ARIMA(asia_ts, order=(1,1,1))
    model_fit = model.fit()

    # Forecast next 5 years
    forecast = model_fit.get_forecast(steps=5)
    forecast_values = forecast.predicted_mean
    conf_int = forecast.conf_int()

    # Plot forecast with improved styling
    plt.figure(figsize=(12, 6))
    plt.plot(asia_ts.index, asia_ts, label='Historical Data', linewidth=2)
    plt.plot(forecast_values.index, forecast_values, color='red',
             linestyle='--', marker='o', label='Forecast')
    plt.fill_between(conf_int.index,
                     conf_int.iloc[:,0],
                     conf_int.iloc[:,1],
                     color='pink', alpha=0.3, label='95% Confidence Interval')

    plt.title('TB Mortality Rate Forecast for Asia (2025-2029)', fontsize=14, pad=2
    plt.ylabel('Mortality Rate (per 100,000)', fontsize=12)
    plt.xlabel('Year', fontsize=12)
    plt.legend(fontsize=12, framealpha=1)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

    # Print forecast values with more context
    print("\n5-Year Mortality Rate Forecast for Asia:")
    forecast_df = pd.DataFrame({
        'Year': forecast_values.index.year,
        'Forecast': forecast_values.round(2),
        'Lower CI': conf_int.iloc[:,0].round(2),
        'Upper CI': conf_int.iloc[:,1].round(2)
```

```
    })
    print(forecast_df.to_string(index=False))

except Exception as e:
    print(f"ARIMA modeling failed: {e}")
```

```
<Figure size 1200x800 with 0 Axes>
```





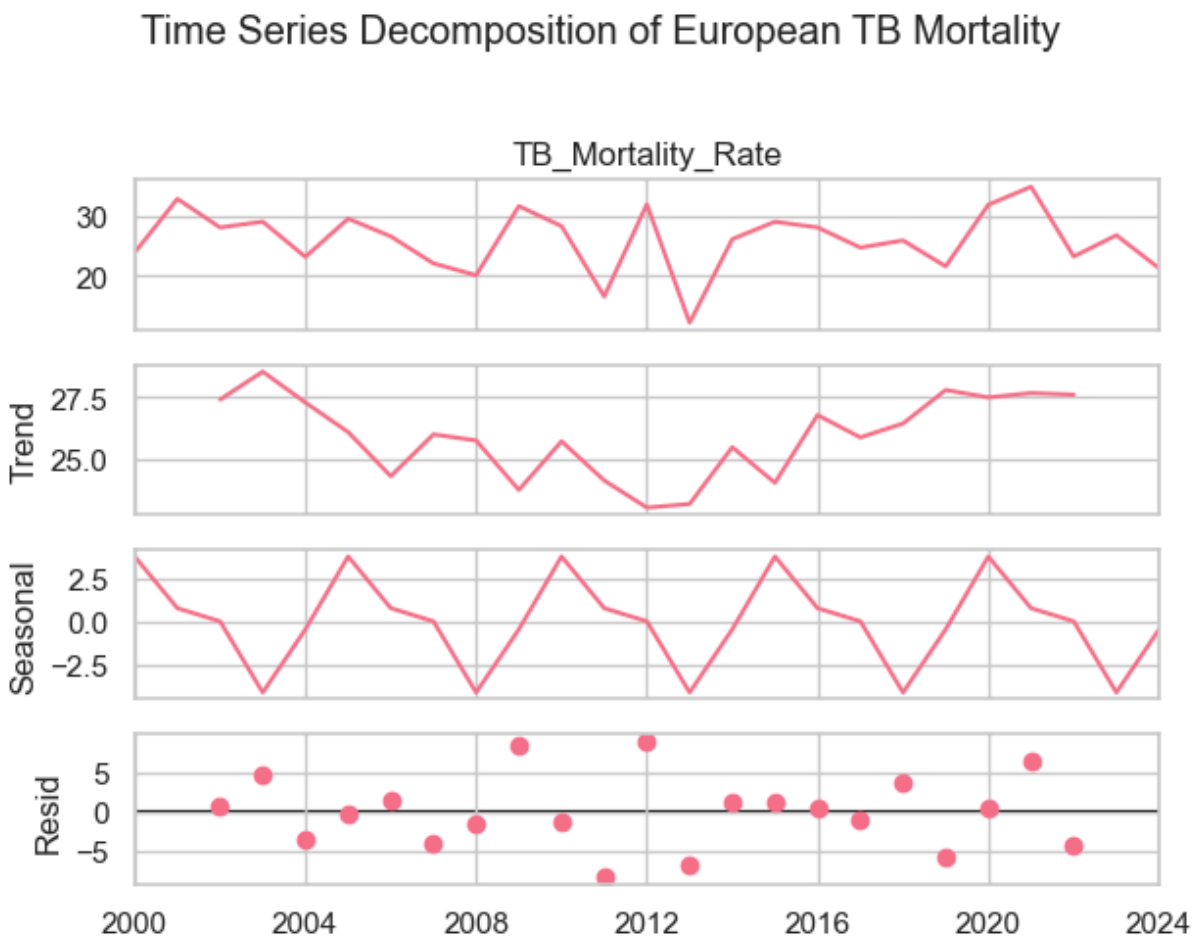
```
5-Year Mortality Rate Forecast for Asia:
 Year  Forecast  Lower CI  Upper CI
 2025     25.20     21.61     28.80
 2026     25.55     21.96     29.15
 2027     25.48     21.81     29.14
 2028     25.49     21.79     29.19
 2029     25.49     21.74     29.23
```

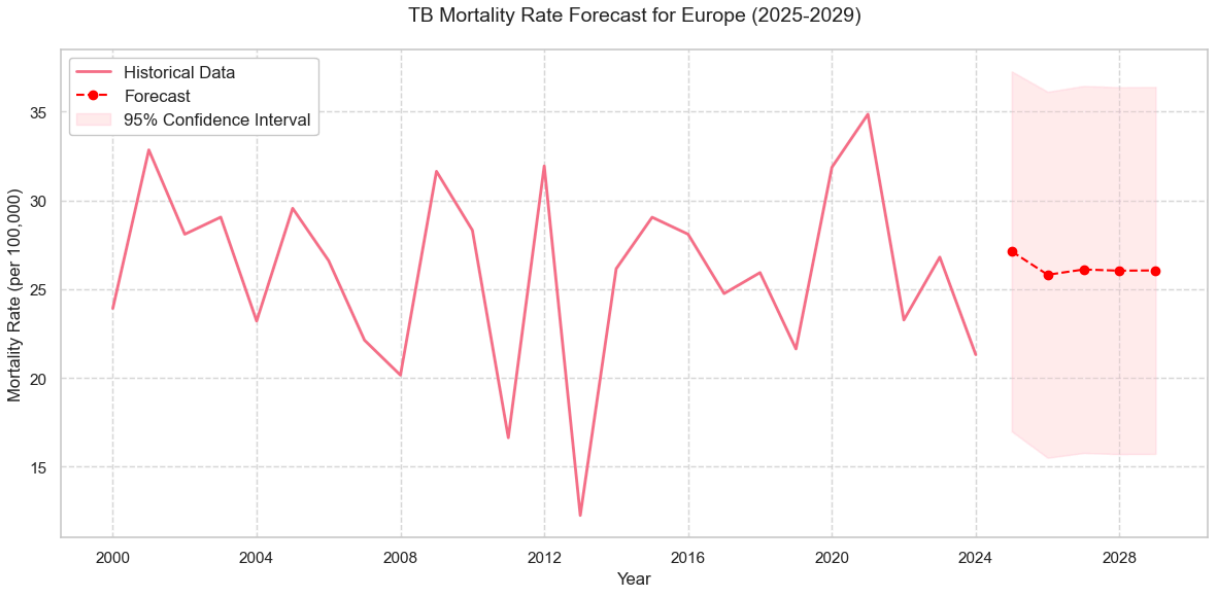
In [42]:

```
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt
import pandas as pd

# Prepare Europe time series with explicit frequency
europe_ts = tb_data[tb_data['Region'] == 'Europe'].groupby('Year')['TB_Mortality_Ra

# Convert to proper time series with yearly frequency
europe_ts.index = pd.to_datetime(europe_ts.index, format='%Y')
europe_ts = europe_ts.asfreq('YS')  # Year-Start frequency

# Time series decomposition
plt.figure(figsize=(12, 8))
try:
    decomposition = seasonal_decompose(europe_ts, model='additive', period=5)
    decomposition.plot()
    plt.suptitle('Time Series Decomposition of European TB Mortality', y=1.02)
    plt.tight_layout()
    plt.show()
except ValueError as e:
    print(f"Decomposition failed: {e}")

# ARIMA modeling with proper frequency handling
try:
    model = ARIMA(europe_ts, order=(1,1,1))
    model_fit = model.fit()

    # Forecast next 5 years
    forecast = model_fit.get_forecast(steps=5)
    forecast_values = forecast.predicted_mean
    conf_int = forecast.conf_int()

    # Plot forecast with improved styling
    plt.figure(figsize=(12, 6))
    plt.plot(europe_ts.index, europe_ts, label='Historical Data', linewidth=2)
    plt.plot(forecast_values.index, forecast_values, color='red',
             linestyle='--', marker='o', label='Forecast')
    plt.fill_between(conf_int.index,
                     conf_int.iloc[:,0],
                     conf_int.iloc[:,1],
                     color='pink', alpha=0.3, label='95% Confidence Interval')

    plt.title('TB Mortality Rate Forecast for Europe (2025-2029)', fontsize=14, pad
    plt.ylabel('Mortality Rate (per 100,000)', fontsize=12)
    plt.xlabel('Year', fontsize=12)
    plt.legend(fontsize=12, framealpha=1)
    plt.grid(True, linestyle='--', alpha=0.7)
```

```
    plt.tight_layout()
    plt.show()

    # Print forecast values with more context
    print("\n5-Year Mortality Rate Forecast for Europe:")
    forecast_df = pd.DataFrame({
        'Year': forecast_values.index.year,
        'Forecast': forecast_values.round(2),
        'Lower CI': conf_int.iloc[:,0].round(2),
        'Upper CI': conf_int.iloc[:,1].round(2)
    })
    print(forecast_df.to_string(index=False))

except Exception as e:
    print(f"ARIMA modeling failed: {e}")
```

```
<Figure size 1200x800 with 0 Axes>
```





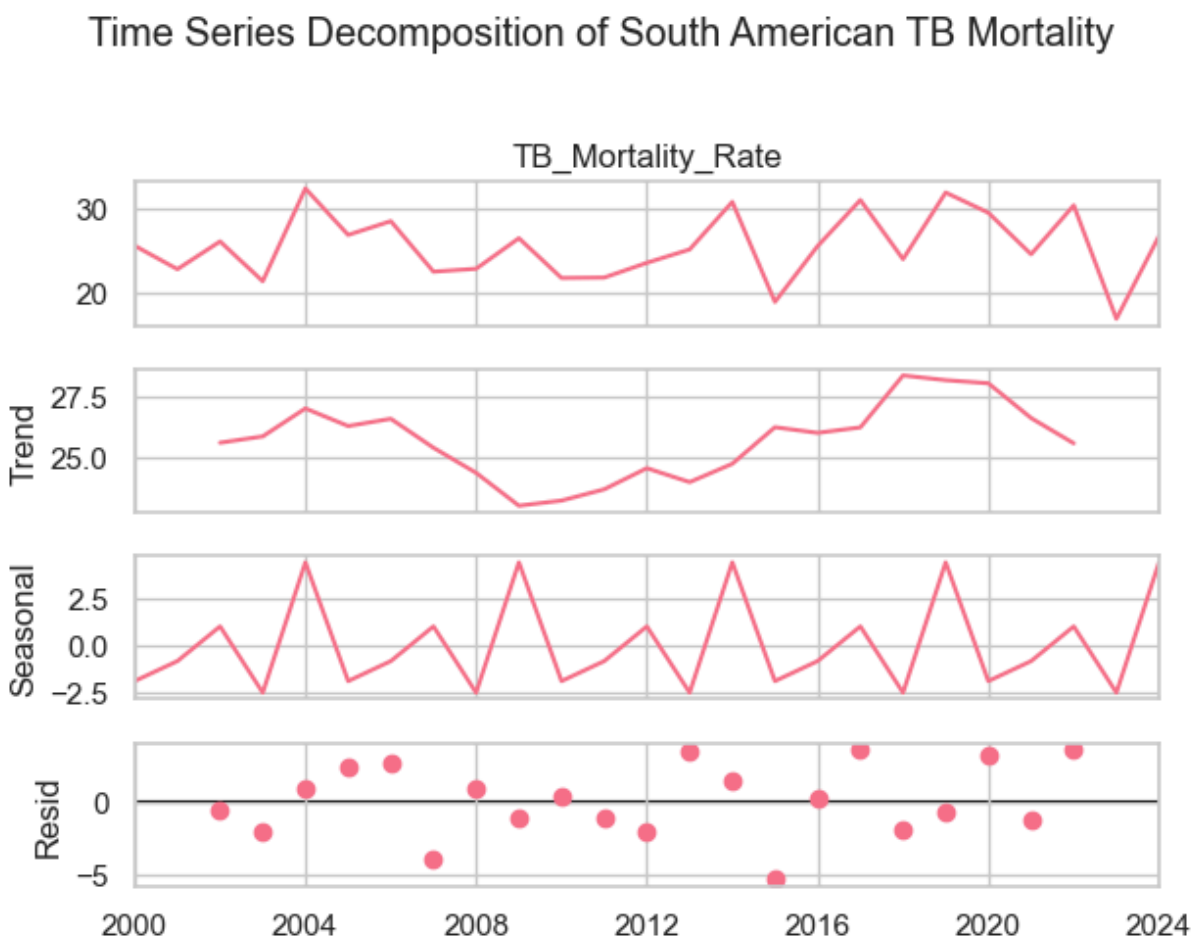

```
5-Year Mortality Rate Forecast for Europe:
 Year  Forecast  Lower CI  Upper CI
 2025     27.12     16.98     37.26
 2026     25.80     15.50     36.11
 2027     26.10     15.77     36.44
 2028     26.04     15.70     36.37
 2029     26.05     15.72     36.38
```

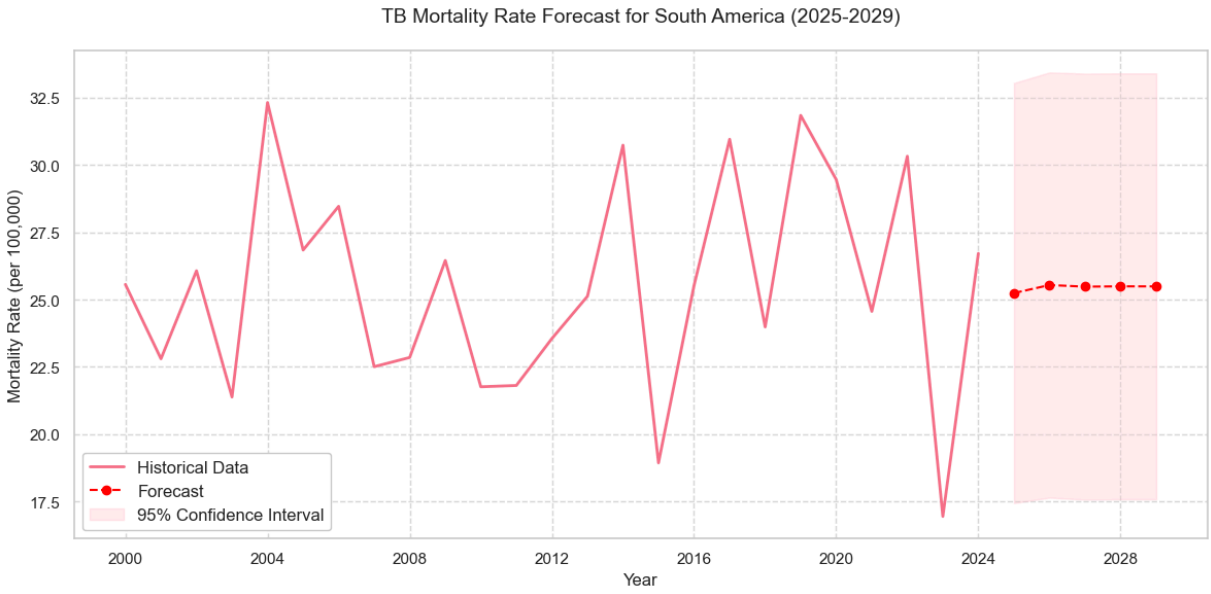
In [17]:

```python
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt
import pandas as pd

# Prepare South America time series with explicit frequency
south_america_ts = tb_data[tb_data['Region'] == 'South America'].groupby('Year')['T

# Convert to proper time series with yearly frequency
south_america_ts.index = pd.to_datetime(south_america_ts.index, format='%Y')
south_america_ts = south_america_ts.asfreq('YS')  # Year-Start frequency

# Time series decomposition
plt.figure(figsize=(12, 8))
try:
    decomposition = seasonal_decompose(south_america_ts, model='additive', period=5
    decomposition.plot()
    plt.suptitle('Time Series Decomposition of South American TB Mortality', y=1.02
    plt.tight_layout()
    plt.show()
except ValueError as e:
    print(f"Decomposition failed: {e}")

# ARIMA modeling with proper frequency handling
try:
    model = ARIMA(south_america_ts, order=(1,1,1))
    model_fit = model.fit()

    # Forecast next 5 years
    forecast = model_fit.get_forecast(steps=5)
```

```
    forecast_values = forecast.predicted_mean
    conf_int = forecast.conf_int()

    # Plot forecast with improved styling
    plt.figure(figsize=(12, 6))
    plt.plot(south_america_ts.index, south_america_ts, label='Historical Data', lin
    plt.plot(forecast_values.index, forecast_values, color='red',
             linestyle='--', marker='o', label='Forecast')
    plt.fill_between(conf_int.index,
                     conf_int.iloc[:,0],
                     conf_int.iloc[:,1],
                     color='pink', alpha=0.3, label='95% Confidence Interval')

    plt.title('TB Mortality Rate Forecast for South America (2025-2029)', fontsize=
    plt.ylabel('Mortality Rate (per 100,000)', fontsize=12)
    plt.xlabel('Year', fontsize=12)
    plt.legend(fontsize=12, framealpha=1)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

    # Print forecast values with more context
    print("\n5-Year Mortality Rate Forecast for South America:")
    forecast_df = pd.DataFrame({
        'Year': forecast_values.index.year,
        'Forecast': forecast_values.round(2),
        'Lower CI': conf_int.iloc[:,0].round(2),
        'Upper CI': conf_int.iloc[:,1].round(2)
    })
    print(forecast_df.to_string(index=False))

except Exception as e:
    print(f"ARIMA modeling failed: {e}")
```

```
<Figure size 1200x800 with 0 Axes>
```





5-Year Mortality Rate Forecast for South America:

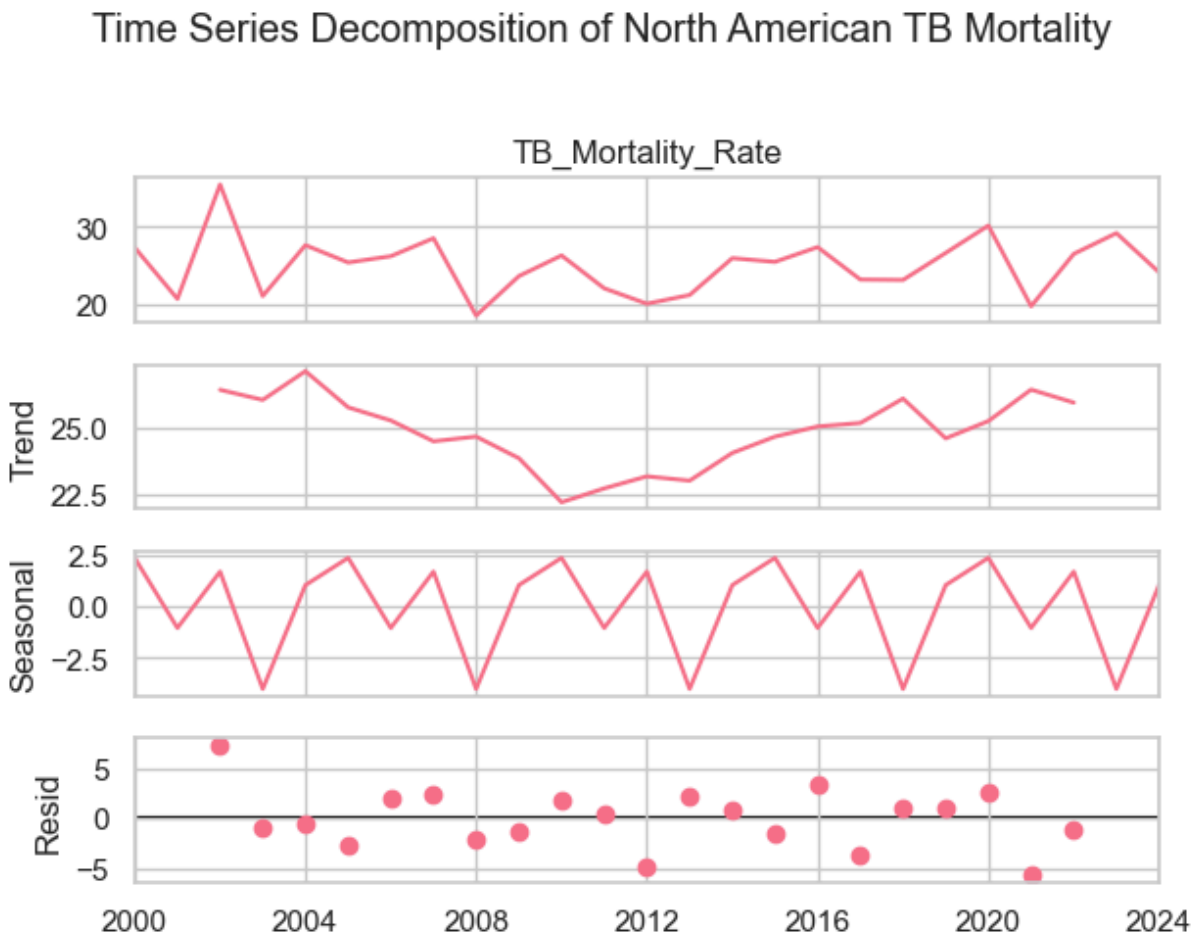
Year	Forecast	Lower CI	Upper CI
2025	25.24	17.44	33.05
2026	25.54	17.64	33.44
2027	25.48	17.56	33.40
2028	25.49	17.58	33.41
2029	25.49	17.57	33.41

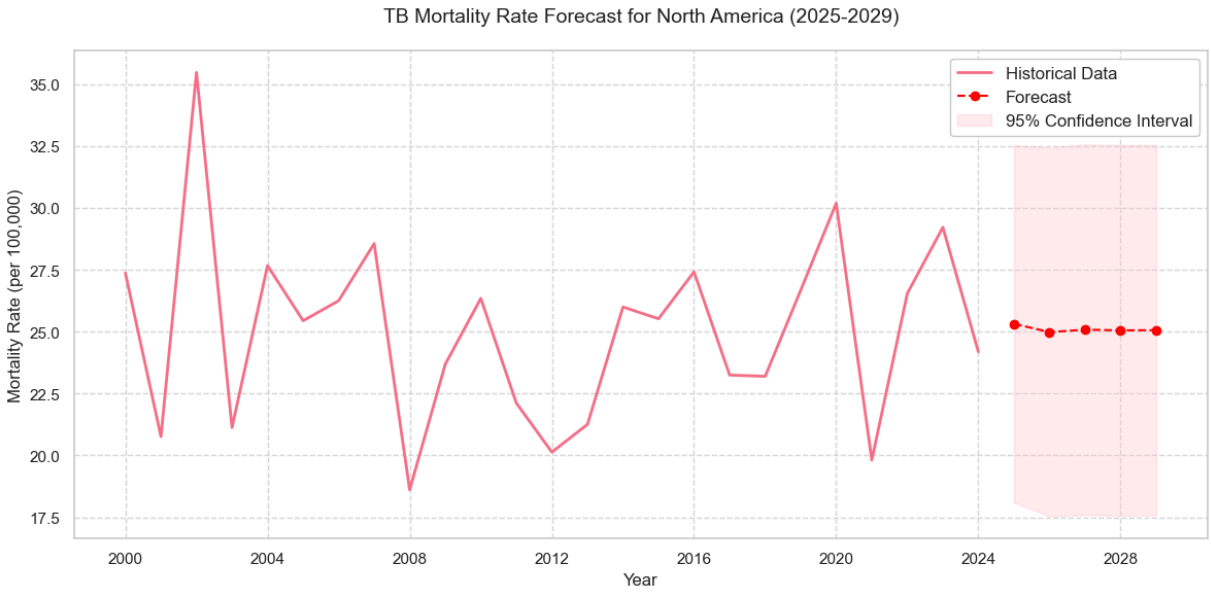
In [32]:
```
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
```

```
import matplotlib.pyplot as plt
import pandas as pd

# Prepare North America time series with explicit frequency
north_america_ts = tb_data[tb_data['Region'] == 'North America'].groupby('Year')['T

# Convert to proper time series with yearly frequency
north_america_ts.index = pd.to_datetime(north_america_ts.index, format='%Y')
north_america_ts = north_america_ts.asfreq('YS')  # Year-Start frequency

# Time series decomposition
plt.figure(figsize=(12, 8))
try:
    decomposition = seasonal_decompose(north_america_ts, model='additive', period=5
    decomposition.plot()
    plt.suptitle('Time Series Decomposition of North American TB Mortality', y=1.02
    plt.tight_layout()
    plt.show()
except ValueError as e:
    print(f"Decomposition failed: {e}")

# ARIMA modeling with proper frequency handling
try:
    model = ARIMA(north_america_ts, order=(1,1,1))
    model_fit = model.fit()

    # Forecast next 5 years
    forecast = model_fit.get_forecast(steps=5)
    forecast_values = forecast.predicted_mean
    conf_int = forecast.conf_int()

    # Plot forecast with improved styling
    plt.figure(figsize=(12, 6))
    plt.plot(north_america_ts.index, north_america_ts, label='Historical Data', lin
    plt.plot(forecast_values.index, forecast_values, color='red',
             linestyle='--', marker='o', label='Forecast')
    plt.fill_between(conf_int.index,
                     conf_int.iloc[:,0],
                     conf_int.iloc[:,1],
                     color='pink', alpha=0.3, label='95% Confidence Interval')

    plt.title('TB Mortality Rate Forecast for North America (2025-2029)', fontsize=
    plt.ylabel('Mortality Rate (per 100,000)', fontsize=12)
    plt.xlabel('Year', fontsize=12)
    plt.legend(fontsize=12, framealpha=1)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

    # Print forecast values with more context
    print("\n5-Year Mortality Rate Forecast for North America:")
    forecast_df = pd.DataFrame({
        'Year': forecast_values.index.year,
        'Forecast': forecast_values.round(2),
        'Lower CI': conf_int.iloc[:,0].round(2),
        'Upper CI': conf_int.iloc[:,1].round(2)
```

```
    })
    print(forecast_df.to_string(index=False))

except Exception as e:
    print(f"ARIMA modeling failed: {e}")
```

```
<Figure size 1200x800 with 0 Axes>
```



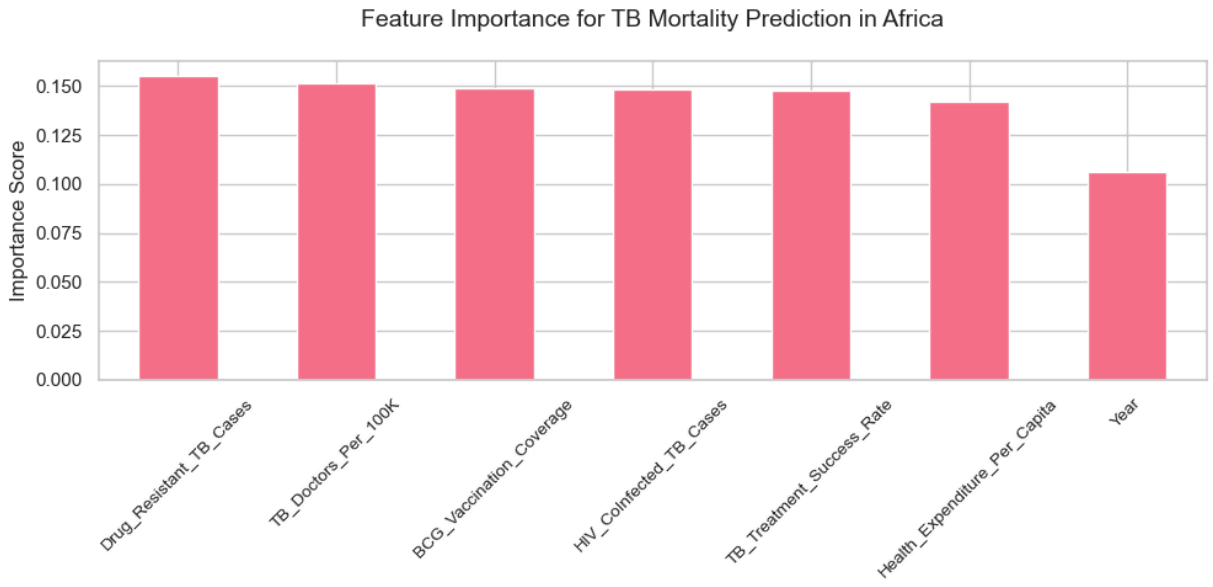


```
5-Year Mortality Rate Forecast for North America:
 Year  Forecast  Lower CI  Upper CI
 2025     25.32     18.10     32.53
 2026     24.98     17.54     32.42
 2027     25.08     17.59     32.57
 2028     25.05     17.57     32.53
 2029     25.06     17.57     32.54
```

In [47]:

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

# Prepare data for ML
africa_data = tb_data[tb_data['Region'] == 'Africa'].copy()
africa_data['Year'] = africa_data['Year'].dt.year  # Convert to numerical year

features = ['Year', 'HIV_CoInfected_TB_Cases', 'Drug_Resistant_TB_Cases',
            'TB_Treatment_Success_Rate', 'Health_Expenditure_Per_Capita',
            'BCG_Vaccination_Coverage', 'TB_Doctors_Per_100K']

X = africa_data[features]
y = africa_data['TB_Mortality_Rate']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Train model
rf = RandomForestRegressor(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)

# Evaluate
y_pred = rf.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("\nModel Performance Metrics:")
print(f"Mean Absolute Error: {mae:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")

# Feature importance
feature_imp = pd.Series(rf.feature_importances_, index=features).sort_values(ascend

plt.figure(figsize=(10, 5))
feature_imp.plot(kind='bar')
plt.title('Feature Importance for TB Mortality Prediction in Africa', fontsize=14,
plt.ylabel('Importance Score', fontsize=12)
plt.xticks(rotation=45, fontsize=10)
plt.tight_layout()
plt.show()
```

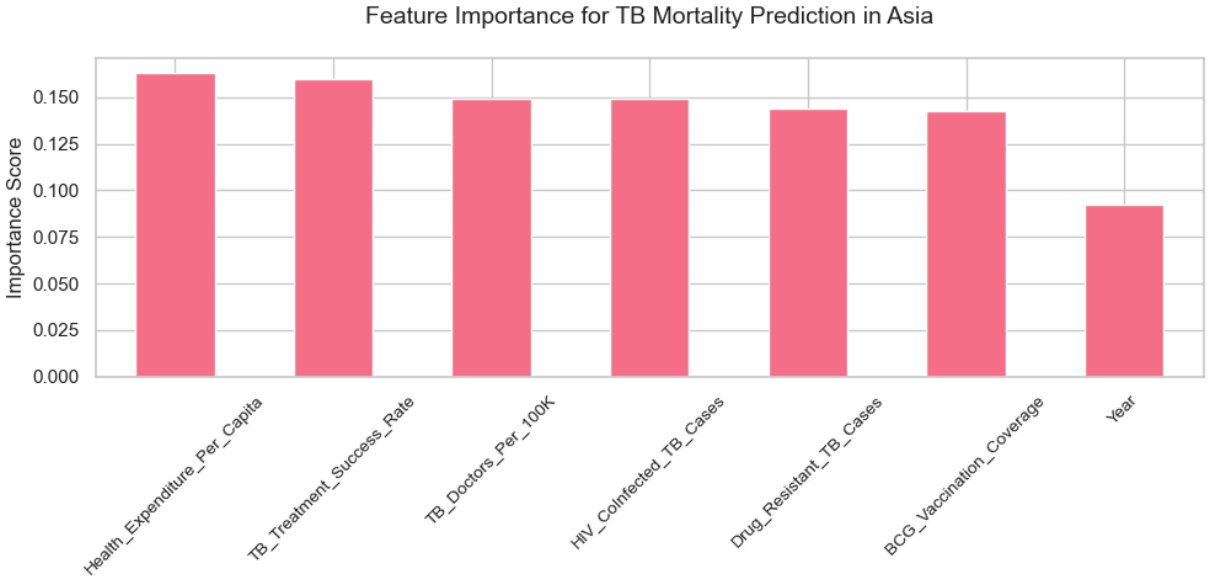
```
Model Performance Metrics:
Mean Absolute Error: 13.07
Root Mean Squared Error: 15.15
```



In [44]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Prepare data for ML - ASIA VERSION
asia_data = tb_data[tb_data['Region'] == 'Asia'].copy()
asia_data['Year'] = asia_data['Year'].dt.year  # Convert to numerical year

features = ['Year', 'HIV_CoInfected_TB_Cases', 'Drug_Resistant_TB_Cases',
            'TB_Treatment_Success_Rate', 'Health_Expenditure_Per_Capita',
            'BCG_Vaccination_Coverage', 'TB_Doctors_Per_100K']

X = asia_data[features]
y = asia_data['TB_Mortality_Rate']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Train model
rf = RandomForestRegressor(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)

# Evaluate
y_pred = rf.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("\nModel Performance Metrics for Asia:")
print(f"Mean Absolute Error: {mae:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")

# Feature importance
feature_imp = pd.Series(rf.feature_importances_, index=features).sort_values(ascend
```

```
plt.figure(figsize=(10, 5))
feature_imp.plot(kind='bar')
plt.title('Feature Importance for TB Mortality Prediction in Asia', fontsize=14, pa
plt.ylabel('Importance Score', fontsize=12)
plt.xticks(rotation=45, fontsize=10)
plt.tight_layout()
plt.show()
```

```
Model Performance Metrics for Asia:
Mean Absolute Error: 11.91
Root Mean Squared Error: 13.87
```

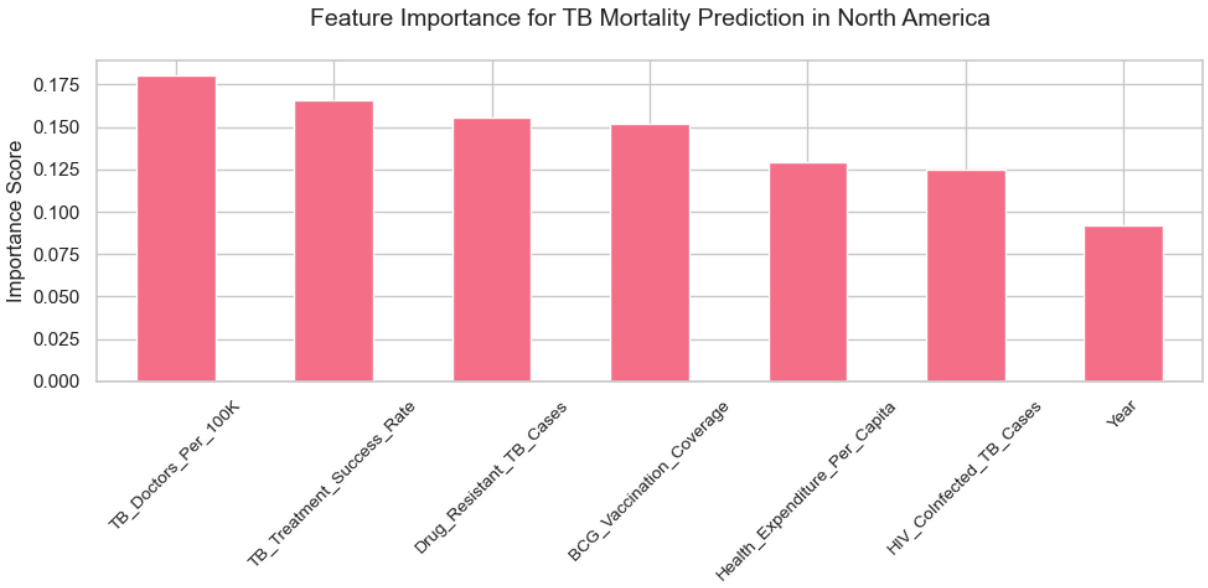


In [23]:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Prepare data for ML - NORTH AMERICA VERSION
north_america_data = tb_data[tb_data['Region'] == 'North America'].copy()
north_america_data['Year'] = north_america_data['Year'].dt.year  # Convert to numer

features = ['Year', 'HIV_CoInfected_TB_Cases', 'Drug_Resistant_TB_Cases',
            'TB_Treatment_Success_Rate', 'Health_Expenditure_Per_Capita',
            'BCG_Vaccination_Coverage', 'TB_Doctors_Per_100K']

X = north_america_data[features]
y = north_america_data['TB_Mortality_Rate']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Train model
rf = RandomForestRegressor(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)

# Evaluate
y_pred = rf.predict(X_test)
```

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("\nModel Performance Metrics for North America:")
print(f"Mean Absolute Error: {mae:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")

# Feature importance
feature_imp = pd.Series(rf.feature_importances_, index=features).sort_values(ascend

plt.figure(figsize=(10, 5))
feature_imp.plot(kind='bar')
plt.title('Feature Importance for TB Mortality Prediction in North America', fontsi
plt.ylabel('Importance Score', fontsize=12)
plt.xticks(rotation=45, fontsize=10)
plt.tight_layout()
plt.show()
```

```
Model Performance Metrics for North America:
Mean Absolute Error: 14.01
Root Mean Squared Error: 16.46
```

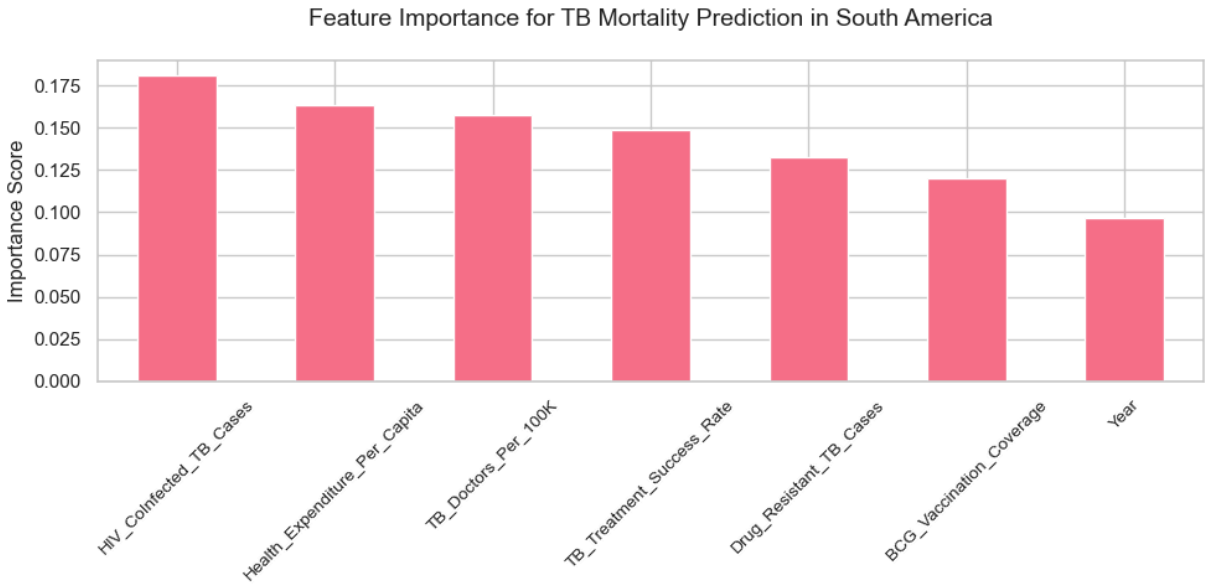


In [45]:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Prepare data for ML - SOUTH AMERICA VERSION
south_america_data = tb_data[tb_data['Region'] == 'South America'].copy()
south_america_data['Year'] = south_america_data['Year'].dt.year  # Convert to numer

features = ['Year', 'HIV_CoInfected_TB_Cases', 'Drug_Resistant_TB_Cases',
            'TB_Treatment_Success_Rate', 'Health_Expenditure_Per_Capita',
            'BCG_Vaccination_Coverage', 'TB_Doctors_Per_100K']

X = south_america_data[features]
```

```
y = south_america_data['TB_Mortality_Rate']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Train model
rf = RandomForestRegressor(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)

# Evaluate
y_pred = rf.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("\nModel Performance Metrics for South America:")
print(f"Mean Absolute Error: {mae:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")

# Feature importance
feature_imp = pd.Series(rf.feature_importances_, index=features).sort_values(ascend

plt.figure(figsize=(10, 5))
feature_imp.plot(kind='bar')
plt.title('Feature Importance for TB Mortality Prediction in South America', fontsi
plt.ylabel('Importance Score', fontsize=12)
plt.xticks(rotation=45, fontsize=10)
plt.tight_layout()
plt.show()
```

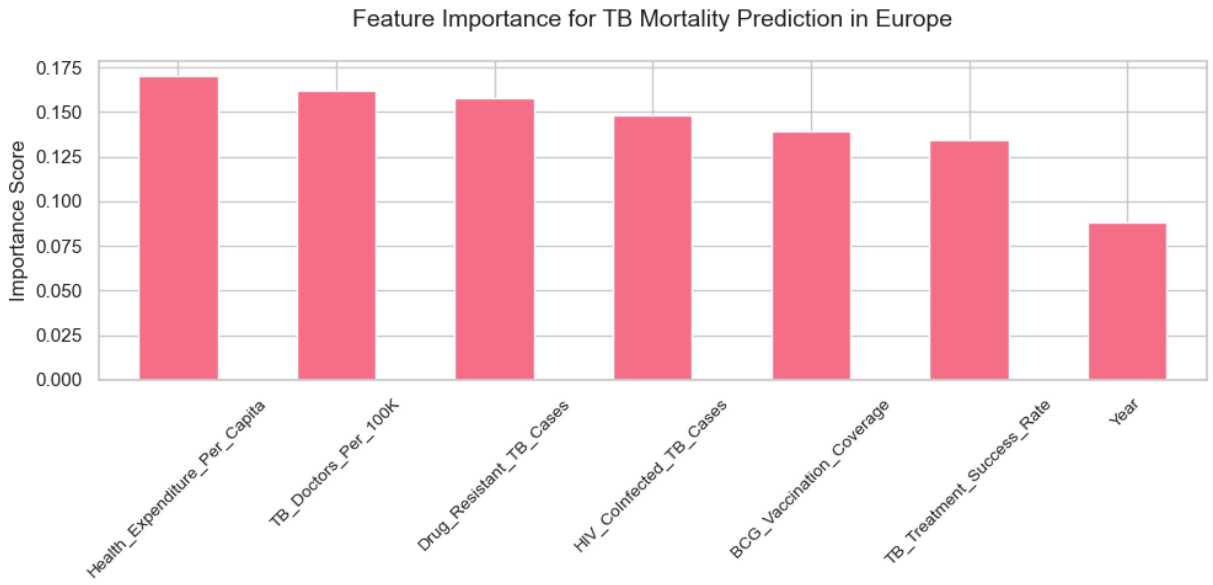
```
Model Performance Metrics for South America:
Mean Absolute Error: 12.43
Root Mean Squared Error: 14.60
```



In [46]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt

# Prepare data for ML - Europe VERSION
europe_data = tb_data[tb_data['Region'] == 'Europe'].copy()
europe_data['Year'] = europe_data['Year'].dt.year  # Convert to numerical year

features = ['Year', 'HIV_CoInfected_TB_Cases', 'Drug_Resistant_TB_Cases',
            'TB_Treatment_Success_Rate', 'Health_Expenditure_Per_Capita',
            'BCG_Vaccination_Coverage', 'TB_Doctors_Per_100K']

X = europe_data[features]
y = europe_data['TB_Mortality_Rate']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Train model
rf = RandomForestRegressor(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)

# Evaluate
y_pred = rf.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("\nModel Performance Metrics for Europe:")
print(f"Mean Absolute Error: {mae:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")

# Feature importance
feature_imp = pd.Series(rf.feature_importances_, index=features).sort_values(ascend

plt.figure(figsize=(10, 5))
feature_imp.plot(kind='bar')
plt.title('Feature Importance for TB Mortality Prediction in Europe', fontsize=14,
plt.ylabel('Importance Score', fontsize=12)
plt.xticks(rotation=45, fontsize=10)
plt.tight_layout()
plt.show()
```

```
Model Performance Metrics for Europe:
Mean Absolute Error: 11.80
Root Mean Squared Error: 13.78
```


Feature Importance for TB Mortality Prediction in Europe
Importance Score
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
Health_Expenditure_Per_Capita
TB_Doctors_Per_100K
Drug_Resistant_TB_Cases
HIV_CoInfected_TB_Cases
BCG_Vaccination_Coverage
TB_Treatment_Success_Rate
Year

In [ ]: